LFX MENTORSHIP PREP

CNCF · Flatcar Container Linux

Cloud-Init to Butane YAML Config Transpiler (Go)

Personal Learning Notes -- 9-Day Bootcamp (Work in Progress)

*These notes cover **Day 1 to Day 7** of an ongoing 9-day self-study bootcamp. Further days will be added to this document as the bootcamp continues -- this is **not** the complete set of bootcamp notes yet.*

Goal	Build a Go-based CLI tool that transpiles Cloud-Init YAML configs into Butane YAML (which Butane itself compiles into Ignition JSON for Flatcar Container Linux).
Why it matters	Thousands of existing servers use Cloud-Init. Flatcar (immutable, container-optimized OS) only understands Ignition/Butane. This tool bridges the two, easing migration.
Covered so far	Day 1 YAML fundamentals · Day 2 Linux provisioning & Cloud-Init intro · Day 3 users/SSH/write_files · Day 4 packages & runcmd · Day 5 Ignition & Butane intro · Day 6 Butane deep dive · Day 7 the Cloud-Init → Butane mapping table
Not yet covered	Day 8 -- Flatcar & project architecture, and Day 9 -- mock interview & application polish -- plus the actual Go implementation, planned for later in the bootcamp.

Skill Map for This Project

Ranked by how critical each skill is to actually shipping the transpiler:

Skill	Depth Needed	Why
Cloud-Init format	**100%**	Input format -- must know every field precisely.
Butane format	**100%**	Output format -- must know every field precisely.
Field mapping logic	**100%**	The actual core of the project -- how each Cloud-Init field converts.
YAML structure/parsing	**80%**	Both formats are YAML; need solid read/write understanding.
Go language basics	**80%**	Implementation language (CNCF standard); no need for advanced Go.
JSON concepts	**80%**	Ignition is JSON; Butane tool auto-generates it -- concept-level is enough.
Linux/provisioning concepts	**40%**	Context for what fields like users/services actually do.
Ignition internals	**40%**	Background only -- the transpiler never touches Ignition directly.

Deliberately not needed for now: advanced Go (concurrency), Kubernetes, actually deploying to cloud/VMs, all 50+ Cloud-Init modules, or all Butane/Ignition edge-case features.

DAY 1

YAML Fundamentals

What is YAML & why it exists

YAML ("YAML Ain't Markup Language") is a human-friendly data serialization format used almost everywhere in cloud-native tooling -- Kubernetes, Docker Compose, Ansible, GitHub Actions, Cloud-Init, and Butane. Compared to XML (verbose) and JSON (no comments, stricter), YAML is readable, supports comments, and needs no braces/quotes for simple values.

Core building blocks

- **Key-value pairs:** `name: kali_user`
- **Data types:** strings, numbers, booleans (true/false/yes/no), null (`null`, `~`, or empty)
- **Lists:** block style with `-`, or flow style `[a, b, c]`
- **Nested structures:** indentation (spaces only, never tabs) defines parent/child relationships
- **Lists of dictionaries:** the most common pattern in Cloud-Init/Butane -- a list where each item is itself a key-value object

Example -- nested structure + list of dictionaries

```
server:
  hostname: web-01
  network:
    ip: 192.168.1.100
    dns_servers:
      - 8.8.8.8
      - 1.1.1.1

users:
  - name: alice
    role: admin
  - name: bob
    role: user
```

Multi-line strings -- critical for scripts

- **|** (literal block) -- **preserves** newlines. Use for bash scripts.
- **>** (folded block) -- **folds** newlines into spaces. Use for long descriptions.

```
startup_script: |
  #!/bin/bash
  echo "starting"
  apt update

description: >
  This text spans several
  lines but becomes one
  folded paragraph.
```

Common mistakes to avoid

- Using **tabs** instead of spaces for indentation (YAML forbids tabs entirely)

- Missing the space after `-` in list items, or after `:` in keys
- Inconsistent indentation between sibling keys
- Unquoted values that look like booleans/numbers when a string was intended (e.g. `version: yes` becomes boolean `true`, not the string "yes")

Tooling

```
# Validate syntax
yamllint file.yaml

# Parse / query values
yq eval '.server.hostname' file.yaml

# Convert YAML -> JSON
yq eval -o=json file.yaml
```

DAY 2

Linux Provisioning & Cloud-Init Introduction

What is "provisioning"?

Provisioning = automatically taking a machine from empty/blank state to a fully configured, ready-to-use state at boot time, instead of manually SSH-ing in and configuring everything by hand.

- **Manual:** slow (30-60 min/server), inconsistent, error-prone, doesn't scale to hundreds of servers.
- **Automated:** one config file gives identical setup applied to any number of servers in minutes.
- **Security angle:** automation means every server gets the exact same hardening (SSH config, firewall, updates) -- no drift, easy to audit.

Declarative vs Imperative

- **Imperative** = describe the exact steps (a bash script). More control, more room for error.
- **Declarative** = describe the desired end-state; the tool figures out how to get there (Cloud-Init, Butane, Kubernetes YAML, Terraform). Idempotent and easier to reason about.

Infrastructure as Code (IaC) -- where Cloud-Init fits

```
Terraform  ->  Cloud-Init / Butane  ->  Ansible (optional)  ->  App deployment
(create VM)    (first-boot config)      (ongoing config mgmt)   (Docker/K8s)
```

Cloud-Init and Butane both operate at the **first-boot configuration** stage.

What is Cloud-Init?

A boot-time provisioning tool, supported by nearly every major cloud provider (AWS, GCP, Azure, DigitalOcean, OpenStack, VMware). You pass it a YAML file as "user-data"; on first boot it creates users, installs packages, writes files, and runs commands -- automatically.

Minimal example

```
#cloud-config
hostname: my-first-server
```

The `#cloud-config` header is **mandatory** -- without it, cloud-init ignores the file entirely.

Top 5 directives most relevant to this project

Field	Meaning
`users`	Create user accounts, set sudo access, shell, groups, SSH keys.
`packages`	List of packages to install (apt/yum, depending on distro).
`write_files`	Create files on disk with specific path/content/permissions/owner.
`runcmd`	Shell commands to run once, after other cloud-init stages, at first boot.
`ssh_authorized_keys`	Public keys allowed to log in for a given user.

Combined example

```
#cloud-config
package_update: true

packages:
  - nginx
  - git

users:
  - name: admin
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    ssh_authorized_keys:
      - "ssh-rsa AAAAB3..."

write_files:
  - path: /etc/motd
    permissions: "0644"
    content: |
      Welcome to the server!

runcmd:
  - systemctl enable nginx
  - systemctl start nginx
```

DAY 3

Cloud-Init Deep Dive: Users, SSH Keys & write_files

`users` -- full field reference

Field	Meaning
`name`	Username (required).
`gecos`	Full name / description (optional).
`shell`	Login shell, e.g. /bin/bash, /bin/false (no login), /usr/sbin/nologin.
`sudo`	true (password required) · ALL=(ALL) NOPASSWD:ALL (full, no password) · false (none) · a specific command path for granular access.
`groups`	Secondary groups list, e.g. [sudo, docker, www-data].
`primary_group`	Primary group name (defaults to a group matching the username).
`ssh_authorized_keys`	List of public SSH keys enabling passwordless login.
`passwd`	A hashed password (never plaintext) -- rarely used, SSH keys preferred.
`lock_passwd`	Whether password login is locked (default true = locked).
`system`	true = system/service account, no home directory, typically no interactive login.

Example -- different access levels in one config

```
users:
  - name: teamlead
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: [sudo, docker]

  - name: junior_dev
    sudo: false
    groups: [developers]

  - name: scanner_bot
    shell: /bin/false
    system: true
    sudo: false
```

SSH keys

- **ED25519** -- modern, fast, fixed 256-bit size. Preferred for new keys.
- **RSA** -- older but universally supported; use at least 4096-bit.
- **DSA** -- deprecated, never use.
- A user can have **multiple** keys listed under `ssh_authorized_keys` (e.g. one per device).
- `ssh_import_id: [gh:username]` can pull public keys directly from GitHub.

```
ssh-keygen -t ed25519 -C "you@example.com"   # generates a new key pair
```

Groups -- primary vs secondary

- **Primary group:** exactly one per user; owns files the user creates.
- **Secondary groups:** zero or more; grant extra permissions (sudo, docker, www-data, adm, etc.).

`write_files` -- full field reference

Field	Meaning
`path`	Absolute destination path (required).
`content`	The file's contents -- use the | block style for scripts.
`permissions`	Octal string in quotes, e.g. "0644" (files), "0755" (scripts), "0600" (secrets).
`owner`	user:group that owns the file, e.g. root:root or www-data:www-data.
`encoding`	text/plain (default) · base64 · gzip · gzip+base64 -- for binary/compressed content.
`append`	true = append to an existing file (e.g. /etc/hosts) · false (default) = overwrite.
`defer`	true = create the file later in the boot process instead of immediately.

Permission conventions to memorize

Field	Meaning
`0755`	rwxr-xr-x -- executable scripts.
`0644`	rw-r--r-- -- regular/public files (default).
`0640`	rw-r----- -- config files with some secrets, group-readable.
`0600`	rw------- -- secrets: passwords, private keys, API tokens.

Example -- script + secret + public file

```
write_files:
  - path: /opt/scripts/start.sh
    permissions: "0755"
    owner: root:root
    content: |
      #!/bin/bash
      systemctl start nginx

  - path: /etc/app/.env
    permissions: "0600"
    owner: appuser:appuser
    content: |
      DB_PASSWORD=supersecret

  - path: /var/www/html/index.html
    permissions: "0644"
    owner: www-data:www-data
    content: |
      <h1>Server Ready</h1>
```

Common mistakes (write_files)

- Relative path instead of absolute (`myfile.txt` instead of `/home/user/myfile.txt`)

- Permissions written without quotes (`0755` instead of `"0755"`) -- YAML may parse it as an octal number, not the intended string.
- Forgetting `"0755"` on scripts -- they simply won't be executable.
- Overwriting `/etc/hosts` with `append: false` instead of appending, losing existing entries.

Early note on the Butane mapping (preview only)

Both `users` and `write_files` map fairly directly onto Butane's `passwd.users[]` and `storage.files[]` sections respectively -- the full field-by-field mapping table is planned for a later day of this bootcamp, once Butane itself has been studied in the same depth as Cloud-Init above.

DAY 4

packages & runcmd Deep Dive

Where Day 3 covered *identity* (users) and *files*, Day 4 covers *software* (packages) and *actions* (runcmd) -- the last two of the four most common Cloud-Init directives.

Restaurant analogy

```
users        = hiring staff
write_files  = menu + SOP documents
packages     = installing oven, fridge, POS machine
runcmd       = "turn on lights, start the oven, run a test order"

-> if the software isn't installed, the commands that depend on it will fail.
   Order matters: packages first, runcmd second.
```

Why `packages` exists (vs just using runcmd)

You *could* write `runcmd: [apt install -y nginx]`, but `packages:` is more declarative -- it states intent ("I need nginx") rather than an exact imperative command, which is easier for a transpiler to reason about.

Core package-related directives

Field	Meaning
`package_update`	Refreshes package manager metadata (like `apt update`). Skipping this can mean outdated info or failed installs.
`package_upgrade`	Upgrades already-installed packages (like `apt upgrade`). Good for security patches, but can add boot time / cause conflicts.
`package_reboot_if_required`	Reboots automatically if an upgrade needs it (e.g. kernel updates).
`packages`	The actual list of packages to install, e.g. `[nginx, git]`.

Version pinning

```
packages:
  - htop
  - [nginx, 1.24.0-1ubuntu1]   # pin an exact version for reproducibility
```

Adding a custom apt repository (concept)

```
apt:
  preserve_sources_list: true
  sources:
    docker.list:
      source: "deb [arch=amd64] https://download.docker.com/linux/ubuntu $RELEASE stable"
      keyid: ABCD1234EFGH5678
package_update: true
packages:
  - docker-ce
```

Flow: default repo doesn't have the package → add a repo → refresh metadata → install.

packages vs runcmd -- separation of concerns

```
packages:
  - nginx

runcmd:
  - systemctl enable nginx
  - systemctl start nginx

# packages = "install the software"
# runcmd   = "now do something with it"
```

Interview gold: packages is a hard mapping area

Flatcar is an **immutable** OS -- there is no native package manager. `packages:` generally has **no direct Butane equivalent**; the typical real-world alternative is running software as a container via a systemd unit instead.

bootcmd vs runcmd

Field	Meaning
`bootcmd`	Runs very early in boot. Low-level prep only -- don't assume network/services/packages are ready yet. Analogy: the worker who opens the shutter at 7am.
`runcmd`	Runs later, once the system is mostly ready. Ideal for systemctl enable/start, running setup scripts, final app config. Analogy: the manager who tests the till at 10am and opens the shop.

runcmd command syntax -- two forms

```
# Form A - plain shell string (shell features like >, |, && work)
runcmd:
  - echo "hello world" > /tmp/hello.txt

# Form B - list form (direct exec, cleaner, but no shell features
#          unless you invoke a shell explicitly)
runcmd:
  - [systemctl, enable, nginx]
  - [sh, -c, 'echo "hello" > /tmp/hello.txt']
```

Good patterns

- **Chain dependent commands** so a failure stops the sequence: `[sh, -c, 'systemctl enable nginx && systemctl start nginx']`
- **Move long logic into a script file** (via write_files) instead of many runcmd lines -- more readable, testable, and easier to migrate to a systemd unit later.
- **Write idempotent commands** where possible, e.g. `mkdir -p`, `ln -sf` -- safe to re-run without breaking the system.

```
write_files:
  - path: /usr/local/bin/app-init.sh
    permissions: "0755"
    content: |
      #!/bin/bash
      set -euo pipefail
      mkdir -p /opt/myapp
```

```
      systemctl restart myapp

runcmd:
  - /usr/local/bin/app-init.sh
```

Common mistakes

- Wrong order: trying to `systemctl start nginx` in runcmd before nginx is even in the `packages` list.
- Non-executable script: content written with `permissions: "0644"` but then executed from runcmd -- needs `"0755"`.
- Using shell redirection (>) inside list-form runcmd -- shell features need `[sh, -c, '...']`, not plain list items.
- Assuming a Flatcar/Butane migration will map `packages` and `runcmd` automatically and cleanly -- these are the two hardest Cloud-Init directives to map.

Day 4 combined example

```
#cloud-config
package_update: true
package_upgrade: false

packages:
  - nginx
  - curl

write_files:
  - path: /var/www/html/index.html
    permissions: "0644"
    content: |
      <html><body><h1>Cloud-Init Deployed Site</h1></body></html>

  - path: /usr/local/bin/post-install.sh
    permissions: "0755"
    content: |
      #!/bin/bash
      set -e
      systemctl enable nginx
      systemctl restart nginx
      echo "Deployment finished on $(date)" >> /var/log/post-install.log

runcmd:
  - /usr/local/bin/post-install.sh
```

DAY 5

Ignition & Butane Introduction

Day 1-4 covered the **input** side of the transpiler (Cloud-Init). Day 5 starts the **output** side: Ignition (the machine format Flatcar actually boots with) and Butane (the human-friendly YAML that compiles into Ignition).

```
Cloud-Init YAML  --> [ YOUR TRANSPILER ] --> Butane YAML --> Ignition JSON
    (INPUT)             (YOU BUILD THIS)        (OUTPUT)      (auto-generated)
```

Analogy

- Cloud-Init = a handwritten recipe in Urdu
- Your tool = the translator
- Butane YAML = the recipe translated to English
- Ignition = the recipe converted into a machine-readable barcode
- Flatcar OS = a robot chef that can only read the barcode

What problem does Ignition solve?

Traditional Cloud-Init provisioning runs multiple stages over 30-60 seconds and can **partially fail** (some steps done, some not). Flatcar's design goal: make provisioning **atomic** -- either the whole config applies, or none of it does -- in 5-10 seconds.

Ignition characteristics

Field	Meaning
Format	JSON (machine-optimized, not meant to be hand-written).
Runs	First boot ONLY -- never re-runs on later boots.
Behavior	Atomic: all-or-nothing. A single failure fails the whole boot.
Speed	~5-10 seconds.
Designed for	Immutable OSes: Flatcar Container Linux, Fedora CoreOS.

Ignition vs Cloud-Init

Feature	Cloud-Init	Ignition
Format	YAML	JSON
Human-writable	Yes	No -- too complex/verbose
Runs when	First boot + later boots	First boot only
Speed	30-60 sec	5-10 sec
Behavior	Best-effort (can partially fail)	Atomic (all-or-nothing)
Package mgmt	Yes (apt/yum)	No
Target OS	Ubuntu / CentOS / etc.	Flatcar / Fedora CoreOS

Sample Ignition JSON (for recognition, not hand-writing)

```
{
  "ignition": { "version": "3.4.0" },
  "passwd": {
    "users": [{
      "name": "admin",
      "sshAuthorizedKeys": ["ssh-ed25519 AAAA... admin@lab"],
      "groups": ["sudo", "docker"]
    }]
  },
  "storage": {
    "files": [{
      "path": "/etc/hostname",
      "contents": { "source": "data:,flatcar-server" },
      "mode": 420
    }]
  }
}
# note: mode 420 (decimal) == 0644 (octal); content is a data: URI.
# nobody wants to hand-write this -- hence Butane.
```

Why Butane exists

```
Human writes:      Butane YAML   (easy)
Butane tool does:  Butane YAML -> Ignition JSON   (automatic)
Flatcar reads:      Ignition JSON   (machine-optimized)
```

Butane YAML skeleton

```
variant: flatcar
version: 1.1.0

passwd:
  users:
    - name: ...

storage:
  files:
    - path: ...
  directories:
    - path: ...

systemd:
  units:
    - name: ...
```

The 4 top-level sections

Field	Meaning
`variant / version`	Mandatory header -- which OS + schema version (like #cloud-config).
`passwd`	Users and groups.
`storage`	Files, directories, and symlinks.
`systemd`	Services / units -- the main way to "run something" on Flatcar.

Cloud-Init users: sudo field -- first mapping challenge

Cloud-Init's `sudo: ALL=(ALL) NOPASSWD:ALL` has **no direct Butane field**. In Butane, sudo access instead comes from putting the user in the `sudo` group:

```
Cloud-Init:  sudo: ALL=(ALL) NOPASSWD:ALL
    -->
Butane:      groups: [sudo]   (+ optionally a sudoers file via storage.files)
```

Cloud-Init write_files -> Butane storage.files (preview)

Cloud-Init	Butane	Notes
`write_files:`	`storage.files:`	different wrapper key
`content:`	`contents.inline:`	restructured
`permissions: "0644"`	`mode: 0644`	quoted string -> integer
`owner: root:root`	`user.name + group.name`	split into two fields

Cloud-Init service pattern -> Butane systemd.units (preview)

Cloud-Init needs 3 separate pieces to run a service: a written unit file (write_files), a daemon-reload, and enable/start (runcmd). Butane combines all of this into **one** `systemd.units` block with `enabled: true`.

Butane CLI

```
butane --pretty --strict config.bu > config.ign
# --pretty  = readable JSON output
# --strict  = treat warnings as errors (safer)

# Your transpiler's output is Butane YAML; the user then runs the
# Butane CLI themselves (or your tool shells out to it) to get Ignition JSON.
```

The big picture

```
1000s of existing Cloud-Init YAML configs
        |
        v
   YOUR TRANSPILER (Go)
   - parse Cloud-Init YAML
   - map each directive
   - generate Butane YAML
   - flag unmappable items
        |
        v
   Valid Butane YAML output
        |
        v
   Butane CLI (existing tool): butane --pretty
        |
        v
   Ignition JSON
        |
        v
   Flatcar Container Linux boots with config
```

Key phrase to remember: **"flag unmappable items"** -- the transpiler doesn't just convert, it also warns about things (like `packages`) that cannot be converted cleanly.

DAY 6

Butane Deep Dive

Butane is the transpiler's **output** format -- if every field isn't understood precisely, the tool will produce output the Butane CLI rejects, and Flatcar simply won't boot.

variant + version

Field	Meaning
`variant`	Target OS: flatcar · fcos (Fedora CoreOS) · openshift · r4e (RHEL for Edge). This project always hardcodes `flatcar`.
`version`	Schema version -- tells the Butane tool which fields are valid. Current for Flatcar: `1.1.0`.

`passwd.users` -- full field reference

Field	Meaning
`name`	Username (required).
`password_hash`	A hashed password string.
`ssh_authorized_keys`	List of public SSH keys.
`uid`	User ID.
`gecos`	Full name / description.
`home_dir`	Home directory path (note: renamed from Cloud-Init's homedir).
`shell`	Login shell.
`groups`	Secondary groups, as a YAML list.
`no_create`	If true, don't create a home directory.
`primary_group`	Primary group name.
`system`	true = system account.
`no_user_group`	Skip auto-creating a group matching the username.
`should_exist`	Ensure this user exists (default true).

Cloud-Init → Butane users mapping

Cloud-Init	Butane	Notes
`name`	`name`	direct
`gecos`	`gecos`	direct
`homedir`	`home_dir`	rename

Cloud-Init	Butane	Notes
`shell`	`shell`	direct
`uid`	`uid`	direct
`primary_group`	`primary_group`	direct
`groups`	`groups`	format change (see below)
`ssh_authorized_keys`	`ssh_authorized_keys`	direct
`passwd (hashed)`	`password_hash`	rename
`system`	`system`	direct
`no_user_group`	`no_user_group`	direct
`lock_passwd`	`no equivalent`	warn
`sudo: ALL=(ALL)...`	`groups: [sudo] + optional sudoers file`	logic (hard)
`expiredate`	`no equivalent`	warn
`inactive`	`no equivalent`	warn
`ssh_import_id`	`no equivalent`	warn
`ssh_redirect_user`	`no equivalent`	warn
`selinux_user`	`no equivalent`	warn

groups format difference

```
Cloud-Init:  groups: sudo, docker            # comma-separated string
Butane:      groups: [sudo, docker]           # YAML list

Transpiler logic: split by comma -> trim spaces -> emit as YAML list.
```

sudo field handling -- critical logic

```
IF Cloud-Init has sudo: ALL=(ALL) NOPASSWD:ALL
THEN  add "sudo" to the Butane groups list
AND   emit a comment noting the original directive

IF Cloud-Init has a command-restricted sudo rule
THEN  optionally create /etc/sudoers.d/<user> via storage.files
AND   emit a warning comment either way
```

passwd.groups

```
Butane:
passwd:
  groups:
    - name: developers
      gid: 2000
    - name: devops
```

`storage.files` -- full field reference

Field	Meaning
`path`	Absolute destination path (required).

`mode`	Permissions as an octal **integer** (not a quoted string).
`overwrite`	Overwrite the file if it already exists?
`user.name / group.name`	File owner user/group, as separate nested fields.
`contents.inline`	Plain-text content, inline in the YAML.
`contents.source`	A `data:` URI -- used for base64/encoded content.
`append`	A list of content blocks to append instead of overwrite.

Cloud-Init → Butane files mapping (detailed)

Cloud-Init	Butane	Notes
`path`	`path`	direct
`content`	`contents.inline`	restructure
`permissions: "0644"`	`mode: 0644`	string -> integer
`owner: root:root`	`user.name + group.name`	split
`encoding: text`	`contents.inline`	direct
`encoding: b64`	`contents.source: data:;base64,...`	transform
`encoding: gzip+b64`	`contents.source: data:;base64,...`	transform
`append: true`	`append: [{inline: ...}]`	restructure
`append: false`	`overwrite: true + contents.inline`	restructure
`defer: true`	`no equivalent`	warn

owner splitting logic

```
Input:  "root:www-data"
Action: split on ":"
Output: user.name = "root", group.name = "www-data"

Edge cases:
  "root"          -> user.name=root, group.name=root (default)
  "appuser:apps"   -> user.name=appuser, group.name=apps
  ""  (empty)      -> skip; use system defaults
```

permissions conversion

```
Input:  "0644"  (quoted string)   -->   Output: mode: 0644  (integer, no quotes)

Common values: 0600 (secrets) - 0640 (config+group read)
            - 0644 (public)  - 0755 (executable) - 0440 (sudoers file)
```

storage.directories & storage.links

Cloud-Init has no dedicated "create a directory" or "create a symlink" field -- these are usually done via `runcmd`: `mkdir -p` / `ln -sf`. Butane has first-class support for both, so the transpiler must **detect these shell patterns** in

runcmd and convert them.

```
storage:
  directories:
    - path: /opt/myapp
      mode: 0755
  links:
    - path: /etc/nginx/sites-enabled/myapp
      target: /etc/nginx/sites-available/myapp
      hard: false

# detected from:
#   runcmd: - mkdir -p /opt/myapp
#   runcmd: - ln -sf /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp
```

`systemd.units` -- the most powerful section

Cloud-Init needs 3 separate steps to manage a service (write the unit file, `daemon-reload`, then `enable` + `start`). Butane collapses all of this into one clean block.

Field	Meaning
`name`	Unit file name, must end in a valid systemd suffix (.service, .timer, .mount, .path).
`enabled`	true = start automatically on every boot (like systemctl enable).
`mask`	true = completely disable the unit, can't even be started manually.
`contents`	The full systemd unit file content ([Unit]/[Service]/[Install] sections).
`dropins`	Override snippets layered on top of an existing unit, without replacing it.

Cloud-Init → Butane systemd mapping

Cloud-Init	Butane	Notes
`write_files: /etc/systemd/system/*.service`	`systemd.units: name + contents`	combine
`runcmd: systemctl enable X`	`systemd.units: enabled: true`	combine
`runcmd: systemctl daemon-reload`	`automatic in Ignition`	remove
`runcmd: systemctl start X`	`covered by enabled: true`	remove
`runcmd: /path/to/script.sh`	`oneshot systemd unit`	create
`cron entry (write_files)`	`.service + .timer pair`	restructure

runcmd script -> oneshot systemd service

```
systemd:
  units:
    - name: setup-app.service
      enabled: true
      contents: |
        [Unit]
        Description=Run setup script on first boot
        After=network-online.target
        Wants=network-online.target
```

```
        ConditionFirstBoot=true

        [Service]
        Type=oneshot
        ExecStart=/opt/scripts/setup-app.sh
        RemainAfterExit=true

        [Install]
        WantedBy=multi-user.target

# Type=oneshot        -> run once and exit, not a daemon
# ConditionFirstBoot  -> mirrors Cloud-Init's "first boot only" behavior
# RemainAfterExit     -> marks the unit "done" after it completes
```

Cron replacement: systemd timer

```
systemd:
  units:
    - name: backup.service
      contents: |
        [Unit]
        Description=Daily Backup
        [Service]
        Type=oneshot
        ExecStart=/opt/scripts/backup.sh
    - name: backup.timer
      enabled: true
      contents: |
        [Unit]
        Description=Run backup daily at 2am
        [Timer]
        OnCalendar=*-*-* 02:00:00
        Persistent=true
        [Install]
        WantedBy=timers.target
```

Day 6 summary

- `variant` + `version` -- always hardcoded to `flatcar` / `1.1.0` for this project.
- `passwd.users` -- watch for groups format, the missing sudo field, and the home_dir/password_hash renames.
- `storage.files` -- watch for mode as integer, owner splitting, and contents.inline vs contents.source.
- `storage.directories` / `links` -- detected from runcmd mkdir/ln patterns.
- `systemd.units` -- combines write_files (service file) + runcmd (enable/start/daemon-reload) into one block; also the target for "run an arbitrary script" via a oneshot unit.

DAY 7

The Cloud-Init → Butane Mapping Table

Day 1-4 covered the input side, Day 5-6 the output side. Day 7 is the **bridge** between them -- effectively the blueprint for the transpiler's core logic, and the answer to the most likely interview question: *"Walk me through how you'd map a Cloud-Init config to Butane."*

Master mapping table -- Users & Groups

Cloud-Init	Butane	Notes
`#cloud-config`	`variant: flatcar / version: 1.1.0`	replace -- easy
`users: / name`	`passwd.users: / name`	direct -- easy
`gecos, shell, uid`	`gecos, shell, uid`	direct -- easy
`homedir`	`home_dir`	rename -- easy
`primary_group, system`	`primary_group, system`	direct -- easy
`groups: "a,b,c"`	`groups: [a, b, c]`	format -- medium
`ssh_authorized_keys`	`ssh_authorized_keys`	direct -- easy
`passwd (hashed)`	`password_hash`	rename -- easy
`sudo: ALL=(ALL)...`	`groups: [sudo] + optional sudoers file`	logic -- hard
`lock_passwd / expiredate / inactive / ssh_import_id / ssh_redirect_user / selinux_user`	`no equivalent`	warn -- N/A
`groups: (top-level)`	`passwd.groups:`	rename -- easy

Master mapping table -- Files, Packages & Hostname

Cloud-Init	Butane	Notes
`write_files:`	`storage.files:`	rename -- easy
`content`	`contents.inline`	restructure -- medium
`permissions: "0644"`	`mode: 0644`	str -> int -- medium
`owner: "root:grp"`	`user.name + group.name`	split -- medium
`encoding: b64 / gzip+b64`	`contents.source: data:;base64,...`	transform -- hard
`append: true/false`	`append: [...] or overwrite: true`	restructure -- medium
`defer`	`no equivalent`	warn -- N/A
`package_update / upgrade / packages / apt:`	`no equivalent`	warn -- N/A
`hostname: myserver`	`storage.files: /etc/hostname (mode 0644, inline)`	transform -- medium

Master mapping table -- runcmd, bootcmd & systemd

Cloud-Init	Butane	Notes
`runcmd: mkdir -p /dir`	`storage.directories`	detect -- medium
`runcmd: ln -s tgt path`	`storage.links`	detect -- medium
`runcmd: systemctl enable X`	`systemd.units (merge)`	merge -- hard
`runcmd: systemctl start X`	`covered by enabled: true`	remove -- medium
`runcmd: systemctl daemon-reload`	`automatic in Ignition`	remove -- easy
`runcmd: /path/script.sh`	`oneshot systemd unit`	create -- hard
`runcmd: arbitrary command`	`oneshot systemd unit + warning`	create -- hard
`bootcmd:`	`oneshot unit with Before= dependency`	create -- hard
`write_files (service file) + runcmd (enable)`	`systemd.units (name+contents+enabled)`	combine -- hard

Master mapping table -- misc directives

Cloud-Init	Butane	Notes
`timezone`	`storage.links: /etc/localtime -> /usr/share/zoneinfo/..`	transform -- medium
`locale`	`no equivalent / storage.files`	warn -- hard
`ssh_pwauth`	`storage.files (sshd_config)`	transform -- hard
`disable_root`	`passwd.users handling`	logic -- medium
`manage_etc_hosts`	`storage.files: /etc/hosts`	transform -- medium
`final_message / phone_home / power_state`	`no equivalent`	skip/warn -- N/A

6 mapping categories

Field	Meaning
1. Direct	Same field, same value -- name, gecos, shell, uid, ssh_authorized_keys. Just copy.
2. Renames	Different field name, same value -- homedir→home_dir, users→passwd.users.
3. Format changes	Same concept, different syntax -- groups string→list, permissions string→mode integer, owner split into user/group.
4. Smart transforms	Requires pattern detection + logic -- hostname, timezone, sudo, encoding.
5. Pattern combination	Multiple Cloud-Init items become one Butane item -- write_files+runcmd→systemd.units, runcmd mkdir/ln→storage.
6. No equivalent	Cannot map -- transpiler must emit a warning: packages, lock_passwd, expiredate, ssh_import_id, defer, final_message, phone_home.

Worked example -- simple user + file

Cloud-Init input

```
#cloud-config
users:
  - name: webadmin
    shell: /bin/bash
    groups: sudo, docker
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA-webadmin-key webadmin@lab

write_files:
  - path: /var/www/html/index.html
    permissions: "0644"
    owner: root:root
    content: |
      <h1>Hello from Flatcar!</h1>
```

Butane output

```
variant: flatcar
version: 1.1.0

passwd:
  users:
    - name: webadmin
      shell: /bin/bash
      groups: [sudo, docker]
      ssh_authorized_keys:
        - ssh-ed25519 AAAA-webadmin-key webadmin@lab
      # NOTE: sudo access granted via 'sudo' group membership
      # Original: sudo: ALL=(ALL) NOPASSWD:ALL

storage:
  files:
    - path: /var/www/html/index.html
      mode: 0644
      user: {name: root}
      group: {name: root}
      contents:
        inline: |
          <h1>Hello from Flatcar!</h1>
```

Worked example -- service + runcmd pattern (packages warned)

Cloud-Init input (key parts)

```
packages: [nginx]

write_files:
  - path: /etc/systemd/system/webapp.service
    content: |
      [Unit]
      Description=Web Application
      [Service]
      ExecStart=/opt/webapp/start.sh
      [Install]
      WantedBy=multi-user.target
```

```
  - path: /opt/webapp/start.sh
    permissions: "0755"
    content: |
      #!/bin/bash
      echo "Starting webapp..."

runcmd:
  - systemctl daemon-reload
  - systemctl enable webapp
  - systemctl start webapp
  - mkdir -p /opt/webapp/logs
  - ln -sf /opt/webapp/start.sh /usr/local/bin/webapp
```

Butane output (key parts)

```
variant: flatcar
version: 1.1.0

# WARNING: 'packages: [nginx]' has no direct Butane equivalent.
# Flatcar is immutable -- consider a container image or systemd unit instead.

storage:
  directories:
    - path: /opt/webapp/logs
      mode: 0755
  files:
    - path: /opt/webapp/start.sh
      mode: 0755
      contents: {inline: |
        #!/bin/bash
        echo "Starting webapp..."}
  links:
    - path: /usr/local/bin/webapp
      target: /opt/webapp/start.sh

systemd:
  units:
    - name: webapp.service
      enabled: true
      contents: |
        [Unit]
        Description=Web Application
        [Service]
        ExecStart=/opt/webapp/start.sh
        [Install]
        WantedBy=multi-user.target
```

Homework (not yet solved in these notes): a third, more complex exercise combining hostname, package_update/upgrade, multiple users (including `lock_passwd`), and a full runcmd sequence -- to be attempted and reviewed on a later day.

Transpiler decision flow (core Go logic)

```
Parse Cloud-Init YAML
        |
        v
FOR each directive:
    #cloud-config?        -> emit variant: flatcar, version: 1.1.0
    hostname?              -> emit storage.files /etc/hostname
    users?                  -> per user: copy/rename direct fields,
```

```
                              split groups string, fold sudo into
                              groups + comment, warn on unsupported
                              fields, emit passwd.users entry
    write_files?            -> if path matches /etc/systemd/system/*.service:
                                 hold for systemd.units merge
                              else:
                                 emit storage.files entry (mode/owner/
                                 contents conversions)
    packages?                -> emit WARNING comment
    runcmd?                   -> per command: mkdir? -> storage.directories
                                 ln -s? -> storage.links
                                 systemctl enable? -> merge enabled:true
                                 systemctl start/daemon-reload? -> skip
                                 script execution? -> oneshot unit
                                 other -> oneshot unit + WARNING
    bootcmd?                   -> oneshot unit with Before= dependency
    unsupported directive?      -> emit WARNING comment
```

This flowchart, sketched on a whiteboard, is a strong way to demonstrate understanding of the whole project in an interview setting.

Day 7 summary

- 6 mapping categories: direct, renames, format changes, smart transforms, pattern combination, no equivalent.
- The transpiler's core logic is: parse → detect pattern → transform or warn → emit output.
- Two worked conversion examples completed above; a third (more complex) exercise is still pending as practice.

End of notes for Day 1-7. This document will be extended as Day 8 (Flatcar & project architecture) and Day 9 (mock interview & application polish) of the bootcamp are completed, along with the actual Go implementation work.